endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 3

AGENDA CURS 3

1. Principiul encapsularii
2. Definirea de mai multe metode cu acelasi nume (supraincarcarea)
3. Refolosirea codului prin mostenirea claselor
4. Redefinirea unei metode mostenite (suprascrierea)
5. Principiul polimorfismului
6. Clase abstracte
7. Interfete

endava

PRINCIPIUL ENCAPSULARII

- Ascunderea detaliilor interne ale unei clase si expunerea doar a unei interfete controlate catre utilizatorii acesteia
- Se realizeaza prin:
  - Declararea atributelor clasei ca private
  - Definirea de metode publice pentru accesarea atributelor (getters si setters)
- Beneficii:
  - Protejeaza datele: previne accesul neautorizat sau modificarile accidentale
  - Imbunatateste mentenabilitatea codului: permite modificari interne fara a afecta utilizatorii clasei
  - Controleaza modul de utilizare a datelor: se pot adauga validari in metodele setter

PRINCIPIUL ENCAPSULARII

```java
class BankAccount {
    private String accountHolder;
    private double balance;

  // Getter for accountHolder
  public String getAccountHolder() {
      return accountHolder;
  }

  // Setter for accountHolder
  public void setAccountHolder(String accountHolder) {
      this.accountHolder = accountHolder;
  }

  // Getter for balance
  public double getBalance() {
      return balance;
  }

 // Simple setter for balance
 public void setBalance(double balance) {
     this.balance = balance;
 }
```


endava

SUPRAINCARCAREA

- Permite definirea mai multor metode cu **acelasi nume**, dar cu **parametri diferiti.**
- Se aplica atat pentru metode (comportamente sau metode statice), cat si pentru constructori.
- Reguli:
  - Metodele trebuie sa aiba acelasi nume, dar parametri diferiti (tip, numar sau ordine diferita a tipului de date).
  - Tipul de return nu conteaza – doar diferentele dintre parametri sunt luate in considerare.
- Beneficii:
  - Creste lizibilitatea – metodele cu acelasi scop au acelasi nume.
  - Flexibilitate – utilizatorul poate apela metoda potrivita nevoilor sale.
  - Imbunatateste reutilizarea codului – evita duplicarea logicii prin metode separate.

SUPRAINCARCAREA

```java
class MathOperations {
    // 1. Overloading prin numar diferit de parametri
    public int multiply(int a, int b) {
        return a * b;
    }

    public int multiply(int a, int b, int c) {
        return a * b * c;
    }

    // 2. Overloading prin tip diferit de parametri
    public double multiply(double a, double b) {
        return a * b;
    }

    // 3. Overloading prin ordine diferita a parametrilor
    public double multiply(int a, double b) {
        return a * b;
    }

    public double multiply(double a, int b) {
        return a * b;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();

        // Apelam metodele supraincarcate
        System.out.println("multiply(int, int): " + math.multiply(2, 3));         // 6
        System.out.println("multiply(int, int, int): " + math.multiply(2, 3, 4)); // 24
        System.out.println("multiply(double, double): " + math.multiply(2.5, 3.5)); // 8.75
        System.out.println("multiply(int, double): " + math.multiply(2, 3.5));   // 7.0
        System.out.println("multiply(double, int): " + math.multiply(2.5, 3));   // 7.5
    }
}
```


endava

MOSTENIREA CLASELOR

- Permite unei clase (subclasa/derivata) sa preia caracteristicile si comportamentele unei alte clase (superclasa/baza)
- Se foloseste cuvantul cheie **extends** pentru a defini o relatie de mostenire intre doua clase.
- Beneficii:
  - Reutilizarea codului – clasele derivate pot folosi atributele si comportamentele din clasa parinte.
  - Reducerea redundantei – evitam rescrierea codului pentru functionalitati similare.
  - Extensibilitate – putem adauga sau modifica functionalitati fara a schimba codul existent.
  - Ofera o structura ierarhica – creeaza relatii de tip "is-a"
- Reguli:
  - O clasa poate mosteni in mod direct doar o singura clasa (in Java nu exista mostenirea multipla a claselor)
  - Se mostenesc doar atributele si comportamentele neprivate
  - Zonele de memorie de clasa si metodele de clasa nu se mostenesc
  - Constructorii nu sunt moșteniți ca comportamentele obișnuite, dar pot fi apelați în subclasă folosind **super(...)**
  - Subclasa poate accesa instanta superclasei folosind **super**
  - Pentru a evita mostenirea unei clase, clasa trebuie marcata cu **final** keyword

SUPRASCRIEREA

- Permite unei subclase sa modifice implementarea unei metode mostenite din superclasa, pastrand aceeasi semnatura (**nume, tip de return, parametri**)
- Metoda care suprascrie trebuie sa respecte urmatoarele reguli:
  - Trebuie sa aiba acelasi nume si aceiasi parametri ca metoda suprascrisa (acelasi numar de parametrii, acelasi tip de data, in aceeasi ordine)
  - Trebuie sa acelasi acelasi tip returnat sau un tip covariant (care poate fi transformat direct in tipul returnat al metodei suprascrise)
  - Trebuie sa aiba aceeasi vizibilitate sau mai mare
  - Nu poate fi private sau static
- Pentru a evita suprascrierea unei metode, metoda trebuie marcata cu **final** keyword
- Se recomanda utilizarea anotarii @**Override** pentru claritate

POLIMORFISM

- Permite ca un obiect sa fie folosit sub mai multe forme: fie sub forma sa explicita, data de clasa care il defineste, fie sub forma oricarei superclase pe care o mosteneste
- Se poate realiza doar in contextual mostenirii
- Permite ca la runtime sa se aleaga metoda care este apelata, in functie de obiectul care se comporta (mechanism facilitate de supracriere)
- Beneficii:
  - Flexibilitate si extensibilitate – codul poate fi modificat fara a afecta alte parti.
  - Permite upcasting-ul (SuperClass obj = new SubClass();) – faciliteaza scrierea de cod generic.
  - Reduce duplicarea codului – o metoda poate avea mai multe implementari fara a crea metode separate.
  - Se integreaza cu mostenirea si suprascrierea pentru a permite personalizarea comportamentului.

POLIMORFISM

```java
class BankAccount {
    public void showAccountType() {
        System.out.println("General Bank Account");
    }
}

class SavingsAccount extends BankAccount {
    @Override
    public void showAccountType() {
        System.out.println("Savings Account");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        BankAccount account1 = new BankAccount();
        BankAccount account2 = new SavingsAccount(); // Upcasting

        account1.showAccountType(); // Apeleaza metoda din BankAccount
        account2.showAccountType(); // Apeleaza metoda suprascrisa din SavingsAccount
    }
}
```


endava

CLASE ABSTRACTE

- O clasa abstracta este o clasa care nu poate fi instantiata direct si este folosita ca baza pentru alte clase
- O clasa abstracta poate contine:
  - Metode abstracte (fara implementare, doar definitie).
  - Metode concrete (cu implementare).
  - Atribute si constructori.
- Se defineste folosind **abstract** keyword
- Poate avea constructori, dar acestia sunt apelati doar prin intermediul subclaselor
- O subclasa a unei clase abstracte trebuie sa implementeze toate metodele abstracte sau sa fie si ea declarata abstracta
- O clasa nu poate fi marcata cu **abstract** si **final** in acelasi timp (nu se poate restrictiona mostenirea si instantierea in acelasi timp)
- Cand sa folosim clase abstracte?
  - Cand avem o structura comuna pentru mai multe clase, dar nu vrem ca aceasta sa fie instantiata direct.
  - Cand unele metode trebuie sa fie obligatoriu implementate in subclase.
  - Cand vrem sa impunem un comportament standard in cadrul unei ierarhii de clase.

CLASE ABSTRACTE

```java
// Clasa abstracta
abstract class Vehicle {
    protected String brand;
    protected int year;

    // Constructor
    public Vehicle(String brand, int year) {
        this.brand = brand;
        this.year = year;
    }

    // Metoda abstracta - fiecare subclasa trebuie sa o implementeze
    public abstract void startEngine();

    // Metoda concreta - poate fi folosita de toate subclasele
    public void displayInfo() {
        System.out.println("Brand: " + brand + ", Year: " + year);
    }
}
```

```java
// Subclasa Car care extinde Vehicle
class Car extends Vehicle {
    private int horsepower;

    public Car(String brand, int year, int horsepower) {
        super(brand, year);
        this.horsepower = horsepower;
    }

    // Implementare metoda abstracta
    @Override
    public void startEngine() {
        System.out.println("Starting the car engine with " + horsepower + " HP.");
    }
}
```


endava

INTERFETE

- O interfata in Java este o colectie de metode abstracte, ce defineste un contract
- Se defineste folosind **interface** keyword
- O clasa poate implementa mai multe interfete, folosind **implements** keyword
- O interfata poate mosteni mai multe interfete, folsind **extends** keyword
- Ce se poate defini intr-o interfata:
  - Zone de memorie constante si statice
  - Metodele sunt implicit publice si abstracte
  - Se pot defini metode cu implementare, folosind modificatorul **default** sau **static**
- Cand sa folosim interfete?
  - Cand vrem sa definim un comportament comun pentru clase care nu au o relatie de mostenire
  - Cand avem nevoie de mostenire multipla
  - Cand vrem sa impunem un contract care vrem sa fie respectat de mai multe clase

INTERFETE

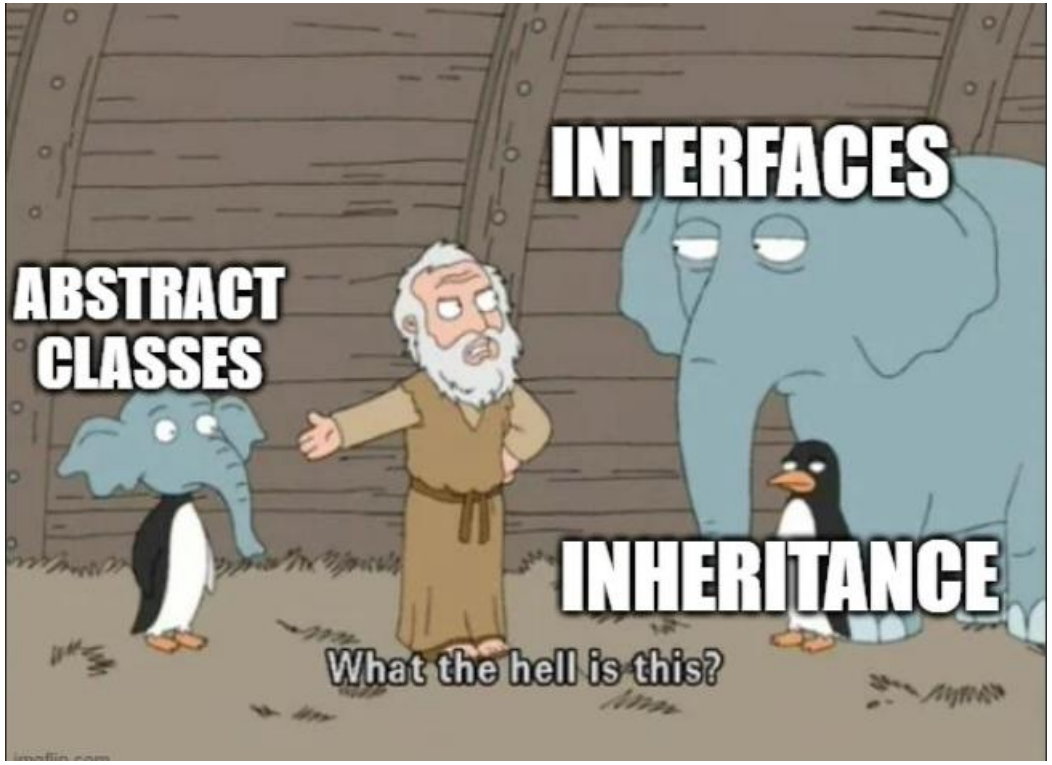
```java
// Definirea unei interfete
interface Device {
    void turnOn();  // Metoda abstracta
    void turnOff();
}

// Implementare a interfetei in clasa Smartphone
class Smartphone implements Device {
    @Override
    public void turnOn() {
        System.out.println("Smartphone is turning on.");
    }

    @Override
    public void turnOff() {
        System.out.println("Smartphone is shutting down.");
    }
}
```


endava

INTERFETE



endava

Q&A

THANK YOU

DANIELA SPILCĂ